

P1664R6

reconstructible_range - a concept for putting ranges back together

Published Proposal, 2022-01-03

Authors:

[JeanHeyd Meneide](#)

[Hannes Hauswedell](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++23

Latest:

https://the-phd.dev/_vendor/future_cxx/papers/d1664.html

Abstract

This paper proposes new concepts to the Standard Library for ranges called Reconstructible Ranges for the purpose of ensuring a range/view broken down into its two iterators can be "glued" back together using an ADL-found function taking a tag, the range's iterator, and the range's sentinel.

Table of Contents

1 Revision History

- 1.1 Revision 6 - January 3rd, 2022
- 1.2 Revision 5 - August 15th, 2021
- 1.3 Revision 4 - June 15th, 2021
- 1.4 Revision 3 - April 15th, 2021
- 1.5 Revision 2 - January 13th, 2020
- 1.6 Revision 1 - November 24th, 2019
- 1.7 Revision 0 - August, 5th, 2019

2 Motivation

- 2.1 Origin
- 2.2 Preservation of Properties
- 2.3 Superior String Splitting
- 2.4 Compile-Time and Interoperability

3 Design

- 3.1 Range? Why not "Borrowed Range"?
- 3.2 Multiple, Safe Reconstruction Points
- 3.3 Opt-in?
- 3.4 Applicability
- 3.5 Why The Overloads?
 - 3.5.1 Case 0: `std::views::split`

3.5.2	Case 1: <code>zstring</code> and <code>std::views::drop</code>
3.5.3	Case 2: Legacy Iterator Code
3.5.4	Case 3: <code>transform_view</code>
3.6	Alternative Design: (<code>range</code> , <code>iterator</code> , <code>sentinel</code>) Reconstruction Point
3.7	Expanding to Owning Containers and More?
4	Deployment Experience
5	Impact
6	Proposed Changes
6.1	Feature Test Macro
6.2	Intent
6.3	Proposed Library Wording
6.3.1	Add a feature test macro <code>__cpp_lib_reconstructible_range</code> .
6.3.2	Insert into §24.2 Header <code><ranges></code> Synopsis [ranges.syn] a new customization point object in the inline namespace:
6.3.3	Insert into §24.4.2 Ranges [range.range]'s after paragraph 7, one additional paragraph:
6.3.4	Insert a new sub-clause "§24.3.13 <code>ranges::reconstruct</code> [<code>range.prim.recons</code>]", after "§24.3.12 <code>ranges::cdata</code> [<code>range.prim.cdata</code>]":
6.3.5	Add to "§21.4.3.1 <i>General</i> " [string.view.template.general] a <i>friend</i> function ADL extension point for <code>basic_string_view</code> :
6.3.6	Add a new subclause "§21.4.  <i>Range Reconstruction</i> " [string.view.reconstruct] in §21.4:
6.3.7	Add to "§22.7.3.1 <i>Overview</i> " [span.overview] a <i>friend</i> function ADL extension point for <code>span</code> :
6.3.8	Add a new subclause "§22.7.3.  <i>Range Reconstruction</i> " [span.reconstruct] in §22.7.3:
6.3.9	Add to "§24.5.4.1 <i>General</i> " [range.subrange.general] a <i>friend</i> function ADL extension point for <code>subrange</code> :
6.3.10	Add a new subclause "§24.5.4.  <i>Range Reconstruction</i> " [range.subrange.reconstruct] in §24.5.4:
6.3.11	Add to "§24.6.4.2 Class template <code>iota_view</code> " [range.iota] a <i>friend</i> function ADL extension point for <code>iota_view</code> :
6.3.12	Add one new paragraph to "§24.6.4.2 Class template <code>iota_view</code> " [range.iota]:
6.3.13	Add to "§24.6.2.2 Class template <code>empty_view</code> " [range.empty.view] a <i>friend</i> function ADL extension point for <code>empty_view</code> :
6.3.14	Add to "§24.7.6.2 Class template <code>transform_view</code> " [range.transform.view] a <i>friend</i> function ADL extension point for <code>transform_view</code> :
6.3.15	Add new paragraphs to "§24.6.4.2 Class template <code>transform_view</code> " [range.transform]:
6.3.16	Modify "§24.7.7.1 Overview " [range.take.overview] for <code>views::take</code> 's paragraph 2:
6.3.17	Modify "§24.7.9.1 Overview " [range.drop.overview] for <code>views::drop</code> 's paragraph 2:
6.3.18	Modify "§24.7.14.2 Class template <code>split_view</code> " [range.split.view] for <code>split_view</code> 's <i>find-next</i> exposition-only function:
6.3.19	Modify "§24.7.14.3 Class <code>split_view::iterator</code> " [range.split.iterator] for <code>split_view<...>::iterator</code> 's <i>value_type</i> in the Synopsis:
6.3.20	Modify "§24.7.14.2 Class template <code>split_view</code> " [range.split.view] for <code>split_view</code> 's Overview and paragraph 5:
7	Acknowledgements
	References
	Informative References

§ 1. Revision History

§ 1.1. Revision 6 - January 3rd, 2022

- Improve the § 2 *Motivation* section, including more explanation which relates to § 2.3 *Superior String Splitting*.
- Trim the customization points slightly and improve the motivation for such points.
- Add case-by-case justifications for the extension points, shown in the § 3.5 *Why The Overloads?* section.

- Vastly improve the wording, especially for [the new split wording](#).
- Include all feedback from SG9 Ranges December 20th, 2021 Virtual Meeting.

§ 1.2. Revision 5 - August 15th, 2021

- Vastly improve wording and address e-mail feedback (thanks, Hannes Hauswedell and Barry Revzin).
- Use the `is_enum_v` and `is_class_v` guards to prevent all bad catch-all (thanks, Christopher Di Bella).
- At the moment, we cannot have the *iterator* and *sentinel* types for things like `iota_view` in the standard-mandated, exposition-only italics, so this little note up here will have to do.
- Ensure that the concepts are properly specified.
- Add lots more motivation to the design, particularly [§ 3.5 Why The Overloads?](#).

§ 1.3. Revision 4 - June 15th, 2021

- Rewrite most of the wording.
- Fix up the design section to always use an ADL Customization Point, all the time. The `is_class_v` and `is_enum_v` protects against most bad cases.

§ 1.4. Revision 3 - April 15th, 2021

- Vastly improve examples.
- Re-target motivation.
- Adjust fixes for things that have changed since C++20.

§ 1.5. Revision 2 - January 13th, 2020

- Improve wording and add more explanation in the design sections for the changes.

§ 1.6. Revision 1 - November 24th, 2019

- Change to `snake_case` concepts. (November 6th)
- Update wording to target [\[n4835\]](#). (November 6th)
- Moved by LWG! 🏆 (November 6th)
- Last minute objection to this paper in plenary.
- Withdrawn; targeting C++23 instead with better design.
- Explored 3 different designs offline: settle on 1 (thanks, Oktal).

§ 1.7. Revision 0 - August, 5th, 2019

- Initial release.

§ 2. Motivation

Below is a quick snapshot of the improvements provided by this paper:

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector<int> vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
span<int> v = s | views::drop(1) | views::take(10)
              | views::drop(1) | views::take(10);
```

✗ - Compilation error

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
auto v = s | views::drop(1) | views::take(10)
           | views::drop(1) | views::take(10);
```

⚠ - Compiles, but `decltype(v)` is `ranges::take_view<ranges::drop_view<ranges::take_view<ranges::drop_view<span<int>, dynamic_extent>>>>>`

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector<int> vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
span<int> v = s | views::drop(1) | views::take(10)
              | views::drop(1) | views::take(10);
```

✓ - Compiles and works with i

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
auto v = s | views::drop(1) | views::take(10)
           | views::drop(1) | views::take(10);
```

✓ - Compiles and works with i
quickcpplib::span<int>

```
char16_t output_buffer[432];
llvm::StringRef input("APLEMI·MYETELIM ჯგ ·[ᄀᄁᄂ ·ᄃ");
llvm::MutableArrayRef<char16_t> output(output_buffer, 432);

auto encoding_result = ztd::text::transcode_into(input, output
          ztd::text::compat_utf8, ztd::text::utf16);
// compiler error here (C++20) !!
llvm::StringRef unprocessed_input
    = encoding_result.input;
llvm::MutableArrayRef<char16_t> unconsumed_output
    = encoding_result.output;
```

✗ - Compilation error

```
std::u8string name = "APLEMI·MYETELIM ჯგ ·[ᄀᄁᄂ ·ᄃ";
char16_t conversion_buffer[432];
std::u8string_view input(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode_into(input, output,
          ztd::text::compat_utf8, ztd::text::utf16);
auto unprocessed_input = encoding_result.input;
auto unconsumed_output = encoding_result.output;
```

⚠ - Compiles, but `decltype(unprocessed_input)` is `ranges::subrange<std::u8string_view::iterator, std::u8string_view::iterator>` and `decltype(unconsumed_output)` is `ranges::subrange<llvm::MutableArrayRef<char16_t>::iterator, llvm::MutableArrayRef<char16_t>::iterator>`

```
char16_t output_buffer[432];
llvm::StringRef input("APLEMI·MYETELIM ჯგ ·[ᄀᄁᄂ ·ᄃ");
llvm::MutableArrayRef<char16_t> output(output_buffer, 432);

auto encoding_result = ztd::text::transcode_into(input, output,
          ztd::text::compat_utf8, ztd::text::utf16);
auto unprocessed_input = encoding_result.input;
auto unconsumed_output = encoding_result.output;
```

✓ - Compiles and works, types

```
std::u8string name = "APLEMI·MYETELIM ჯგ ·[ᄀᄁᄂ ·ᄃ";
char16_t conversion_buffer[432];
std::u8string_view input(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode_into(input, output,
          ztd::text::compat_utf8, ztd::text::utf16);
auto unprocessed_input = encoding_result.input;
auto unconsumed_output = encoding_result.output;
```

✓ - Compiles and works, where
`std::u8string_view` and `decltype(unconsumed_output)` is `std::span<char16_t>`.

```
char const* words = "A quick brown fox";
for (std::span ss : zstring{words} | std::views::split(' ')) {
    auto sv = std::string_view(ss.data(), ss.size());
    std::cout << sv << '\n';
}
```

```
char16_t output_buffer[432];
llvm::StringRef input("APLEMI·MYETELIM ჯგ ·[ᄀᄁᄂ ·ᄃ");
llvm::MutableArrayRef<char16_t> output(output_buffer, 432);

auto encoding_result = ztd::text::transcode_into(input, output,
          ztd::text::compat_utf8, ztd::text::utf16);
auto unprocessed_input = encoding_result.input;
auto unconsumed_output = encoding_result.output;
```

✗ - Compilation error

```
std::u8string name = "AKLEMI·M̄ETELIM ზღ ·[ᄇᄃ ᄃᄂ";
char16_t conversion_buffer[432];
std::u8string_view input(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode_into(input, output,
    ztd::text::compat_utf8, ztd::text::utf16);
auto unprocessed_input = encoding_result.input;
auto unconsumed_output = encoding_result.output;
```

⚠ - Compiles, but `decltype(unprocessed_input)` is `ranges::subrange<std::u8string_view::iterator, std::u8string_view::iterator>` and `decltype(unconsumed_output)` is `ranges::subrange<llvm::MutableArrayRef<char16_t>::iterator, llvm::MutableArrayRef<char16_t>::iterator>`

```
= encoding_res
llvm::MutableArrayRef<
= encoding_res
```

✓ - Compiles and works, types

```
std::u8string name = "
char16_t conversion_bu
std::u8string_view inp
std::span<char16_t> ou

auto encoding_result =
    ztd::text::utf
auto unprocessed_input
auto unconsumed_output
```

✓ - Compiles and works, where `std::u8string_view` and `decltype(unconsumed_output)` are `std::span<char16_t>`.

```
std::string_view s = "12.22.32.41";

auto ints =
    s | views::split('.')
    | views::transform([](auto v){
        int i = 0;
        // only the numbers ending in 2
        if (v.ends_with('2')) {
            std::from_chars(v.data(), v.size(), &i);
        }
        return i;
    });
```

✗ - Compilation error (`.starts_with(...)` is not present on `v` which is an object of type `std::ranges::subrange<const char*, const char*, std::ranges::subrange_kind::sized>`).

```
std::string_view s = "

auto ints =
    s | views::spl
    | views::tra
        int i
        // onl
        if (v.
    }
    return
```

✓ - Compiles and works, as expected

§ 2.1. Origin

This paper originally began as an attempt to handle the fact that certain views/ranges, once pulled apart into their iterator/sentinel, could not be put back together in any logically consistent manner. Currently in C++, there is no Generic ("with a capital G") way to take a range apart with its iterators and put it back together. That is, the following code is not guaranteed to work:

```
template <typename Range>
auto operate_on_and_return_updated_range (Range&& range) {
    using I = std::ranges::iterator_t<Range>;
    using S = std::ranges::sentinel_t<Range>;
    using Result = std::remove_cvref_t<Range>;
    if (std::ranges::empty(range)) {
        // ⚠!
        // ... the below errors
        return Result(std::forward<Range>(range));
    }
    /* perform some work with the
    iterators or similar */
    auto first = std::ranges::begin(range);
    auto last = std::ranges::end(range);
    if (*first == u'\0xEF') {
        // ...
        std::advance(first, 3);
        // ...
    }
}
```

```

    }
    // ... algorithm finished,
    // return the "updated" range!

    // 🚫!
    // ... but the below errors
    return Result(std::move(first), std::move(last));
}

int main () {
    std::string_view meow_view = "나는 유리를 먹을 수 있어요. 그래도 아프지 않아요";
    // this line will error in C++17 and C++20, or in compatibility
    // libraries
    std::string_view sub_view
        = operate_on_and_return_updated_range(meow_view);
    return 0;
}

```

While all of the container types and a decent chunk of current-day views have constructors that take `iterator`, `iterator` pairs or something convertible from those, it was never actually a formal concept. It was recommended to find a better [concept to model this property](#). In the meantime, the current fix is to employ `std::ranges::subrange<I, S, K>` to return a generic subrange:

```

template <typename Range>
auto operate_on_and_return_updated_range (Range&& range) {
    using I = std::ranges::iterator_t<Range>;
    using S = std::ranges::sentinel_t<Range>;
    using Result = std::ranges::subrange<I, S>;
    if (std::ranges::empty(range)) {
        return Result(std::forward<Range>(range));
    }
    // perform some work with the
    // iterators or similar
    auto first = std::ranges::begin(range);
    auto last = std::ranges::end(range);
    if (*first == u'\0xEF') {
        // ...
        std::advance(first, 3);
        // ...
    }
    // ... algorithm finished,
    // return the "updated" range!

    // now it works!
    return Result(std::move(first), std::move(last));
}

int main () {
    std::string_view meow_view = "나는 유리를 먹을 수 있어요. 그래도 아프지 않아요";
    auto sub_view
        = operate_on_and_return_updated_range(meow_view);
    // decltype(sub_view) ==
    //   std::ranges::subrange<std::string_view::iterator, std::string_view::iterator>
    //   which is nowhere close to ideal.
    return 0;
}

```

This makes it work with any two pair of iterators, but quickly becomes undesirable from an interface point of view. If a user passes in a `quickcpplib::span<T, Extent>` or a `llvm::StringRef` that interface and information is entirely lost to the user of the above function. `std::ranges::subrange<Iterator, Sentinel, Kind>` does not -- and cannot/should not -- mimic the interface of the view it was created from other than what information comes from its iterators: it is the barebones idea of a pair-of-iterators/iterator-sentinel style of range. This is useful in the generic sense that

if a library developer must work with iterators, they can always rely on creation of a `std::ranges::subrange` of the iterator and sentinel.

§ 2.2. Preservation of Properties

Unfortunately, always using `std::ranges::subrange` decreases usability for end users. Users who have, for example a `llvm::StringRef`, would prefer to have the same type after calling an algorithm. These types have functions and code that are purpose-made for `string_view`-like code: losing that intent means needing to reconstruct that from the ground up all over again. There is little reason why the original type needs to be discarded if it supports being put back together from its iterators.

The break-it-apart-and-then-generic-`subrange` approach also discards any range-specific storage optimizations and layout considerations, leaving us with the most bland kind of range similar to the "pair of iterators" model. Consider, for example, `llfio::span<T>`. The author of the Low Level File IO (LLFIO) library, Niall Douglass, has spent countless days submitting pull requests and change requests to MSVC, GCC, and Clang's standard libraries to ask them to change their structural layout to match things like the `iovec` of their platform or their asynchronous buffer span types. This optimization matters because it realizes the performance difference between having to individually transformed a container of `spans` to a more suitable type, or being able to simply `memcpy` the desired sequences of spans into the underlying asynchronous operations. If range operations were to be performed on the `llfio::span<T>` type, a `std::ranges::subrange` would hold, in most cases, two iterators rather than an iterator and a size. This completely eliminates the `memcpy` optimization. This is not the only place this happens, however: other types have different storages requirements that are not appropriately captured by their iterators in the most general sense, but may benefit from reconstruction and desired type information for the target range they should be constructed into.

§ 2.3. Superior String Splitting

[Superior String Splitting](#) is a paper for having both an eager and a lazy split view in the standard library. It argues that because we have a maximally lazy split and always present a forward-range, we should separate the always-input-range functionality of the current split into `std::ranges::lazy_split*` by renaming, and then add more-eager `std::ranges::split` that takes advantage of the forward-only semantics to upgrade the capabilities of the type and always return a `std::ranges::subrange<std::iterator_t<V>>`.

One of the questions it brings up is notable:

For splitting a string, this means we get a range of `subrange<string::iterator>` where we might wish we got a `span<char const>` or a `string_view`, but `span<char const>` is already constructible from `subrange<string::iterator>` and `string_view` would be with the adoption of [\[P1989R0\]](#).

We could additionally favor the char case (since, again, splitting strings is the overwhelmingly common case)...

... But this just seems weirdly specific and not a good idea.

— Barry Revzin, [p2210r2 "\\$3.1 What should the reference type be?"](#)

Revzin notes that string splitting here could return something better than a `std::ranges::subrange` based on the input. However, such a proposition in the current C++20 landscape is tenuous at best because it would require adding special cases to the split requirements. The below design comes as a generic way to provide "a reconstructed or related range if possible, otherwise a `subrange`". This would meet the requirements of Superior String Splitting while allowing for better return types when an `llvm::StringRef`, `llvm::ArrayRef`, `TArrayView` from Unreal Engine, `std::wstring_view`, or `quickcpplib::span` is used as the input range.

§ 2.4. Compile-Time and Interoperability

Compilation time goes up as well in the current paradigm. Users must spawn a fresh `std::ranges::subrange<I, S, K>` for every different set of iterator/sentinel/kind triplet, or handle deeply nested templates in templates as the input types. This makes it impossible to compile interfaces as dynamic libraries without having to explicitly materialize or manually cajole a `std::ranges::subrange` into something more palatable for the regular world.

There is also a problem where there are a wide variety of ranges that could conceivably meet this criterion, but do not. The author of this paper was not the only one to see utility for this. [p1739r4] does much the same that this paper does, without the introduction of a concept to formalize the behavior it presents. In particular, it selects views which can realistically have their return types changed to match the input range and operations being performed (or a similarly powerful alternative) by asking whether they can be called with a function called `reconstruct` from a subrange of the iterators with the expressions acted upon.

- Ranges should be reconstructible from their iterators where applicable;
- and, reconstructible ranges serve a useful purpose in generic algorithms, including not losing information and returning it in a much more cromulent and desirable form.

Therefore, this paper formalizes that concept and provides an **opt-in, user-overridable way** to return a related "reconstructed" type from an tag, an iterator, and a sentinel (or more/less, depending on the necessary situation).

§ 3. Design

The design is given in 3 concepts added to the standard:

```
template <class It, class Sen = It>
concept iterator_reconstructible_range =
(std::is_class_v<It> || std::is_class_v<Sen> || std::is_enum_v<It> || std::is_enum_v<Sen>)
&& requires (It first, Sen last) {
    reconstruct(
        std::forward<It>(first),
        std::forward<Sen>(last)
    );
};

template <class R,
    class It = ranges::iterator_t<R>,
    class Sen = ranges::sentinel_t<R>>
concept reconstructible_range =
std::ranges::range<R>
&& (iterator_reconstructible_range<It, Sen>
|| requires (It first, Sen last) {
    reconstruct(
        in_place_type<remove_cvref_t<R>>,
        std::forward<It>(first),
        std::forward<Sen>(last)
    );
});

template <class R, class Tag = remove_cvref_t<R>,
    class It = ranges::iterator_t<R>,
    class Sen = ranges::sentinel_t<R>>
concept range_iterator_reconstructible_range =
std::ranges::range<R>
&& (reconstructible_range<Tag, It, Sen>
|| requires (R range, It first, Sen last) {
    reconstruct(
        in_place_type<Tag>,
        std::forward<R>(range),
        std::forward<It>(first),
        std::forward<Sen>(last)
    );
});
```

It is the formalization that a range can be reconstructed from its necessary components. The concepts are a means of implementing the desired Customization Point Object (CPO) which will be called `std::ranges::reconstruct`, and for checking if a reconstruction point will actually be called rather than falling back to a `std::ranges::subrange<...>`

return. The 3 forms allow for 3 levels of construction, where each one defers to the next version by removing one layer of "information" until finally following back to the most basic form:

- the desired type to reconstruct ("tag"), the range, the iterator, and the sentinel;
- the desired type to reconstruct ("tag"), the iterator, and the sentinel, after stripping out the range if the previous `reconstruct` did not find anything;
- the iterator and the sentinel, after dropping the tag if the previous `reconstruct` attempt did not work and making sure at least one of `std::is_class_v<iterator>`, `std::is_class_v<sentinel>`, `std::is_enum_v<iterator>`, or `std::is_enum_v<sentinel>` is `true`; and finally,
- `std::ranges::subrange<decltype(iterator), decltype(sentinel)>` if nothing else matches.

This allows a developer to put a range back together at various levels of fidelity from its parts and pieces, giving us the ability to propagate the input type's properties after modifying its iterators for some underlying work, algorithm or other effect. This concept is also the basis of the idea behind [p1739r4], which was accepted in a lesser form for C++20 with the intent of this paper following up on it. Algorithm authors can use the extension point and the concepts to simplify their code or to add support for a broader variety of types: see [further down in this proposal](#) for an example of a possible use case for making `std::views::split` more robust in the presence of containers.

Note that, when defining a reconstruction point, a normal user is NOT required to override all three potential customizations. A user may override one form, two forms, or all three forms, depending on what kind of information may be available to help make the best decision for reconstruction possible.

§ 3.1. Range? Why not "Borrowed Range"?

Previously, we required that the ranges being reconstructed modeled `borrowed_range`, since that was an accurate indicator that the iterators and sentinel could potentially outlive the range itself. This was to prevent some issues in a previous design that used constructors. Since that is no longer based on constructors and with no chance of false positives, we no longer need to include that limitation. `borrowed_range` is also too restrictive, as there are some cases where the resulting range is non-borrowed, but could be successfully reconstructed from some of its pieces. The concept is opt-in now by way of declaring an ADL-found `reconstruct` function, any type can effectively decide for itself whether it could reconstruct properly. Note that this opens up usages for things that, if we had used `borrowed_range`, would not have been reconstructible. Consider the following expression:

```
int f (int v) {
    return v * 2;
}

int main () {
    std::vector<int> vec{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    std::span<int> some_span(vec);
    auto transform = some_span
        | std::views::transform(f)
        | std::views::take(5);
    // ...
    return 0;
}
```

This particular case is reconstructible as `span<T>` is reconstructible. The original transformation view can be reconstructed here without the need to keep the `take_view` intermediate wrapped around the two previous transformations, as the transform can be ignored on the first five elements by advancing the given `some_span`'s `begin()` by 5 elements. A smart `reconstruct` call for `take_view` does this (and is present in [§ 6.3 Proposed Library Wording](#)).

§ 3.2. Multiple, Safe Reconstruction Points

Consider a hypothetical `c_string_view` type. It is impossible to, from normal `const char*` iterator pair ascertain the range is null-terminated and is thus a "C string". Therefore, a reconstruct for `c_string_view` would not return a `c_string_view`, but a normal `string_view`.

On the other hand, if there was a `c_string_sentinel` sentinel with a `const char*` iterator that was preserved through a series of actions and algorithms, that kind of information would allow us to return a `c_string_view` from a reconstruction operation.

The `reconstruct` extension point supports both of these scenarios:

```
class c_string_view {
    /* blah... */

    // reconstruct: with [const char*, const char*) iterators
    // no guarantee it's null terminated: decay to string_view
    friend constexpr std::string_view reconstruct (
        std::in_place_type_t<c_string_view>,
        const char* first, const char* last) noexcept {
        return std::string_view(first, last);
    }

    // reconstruct: with static [const char*, c_string_sentinel)
    // static guarantee by type: return a c_string_view
    friend constexpr c_string_view reconstruct (
        std::in_place_type_t<c_string_view>,
        const char* first, c_string_sentinel) noexcept {
        return c_string_view(first);
    }
};
```

This is a level of flexibility that is better than the Revision 0 constructor-based design, and can aid in providing better static guarantees while decaying gracefully in other situations.

§ 3.3. Opt-in?

Not all ranges can meet this requirement. Some ranges contain state which cannot be trivially propagated into the iterators, or state that cannot be reconstructed from the iterator/sentinel pair itself. However, most of the common ranges representing unbounded views, empty views, iterations viewing some section of non-owned storage, or similar can all be reconstructed from their iterator/iterator or iterator/sentinel pair.

For example `std::ranges::single_view` contains a [exposition-only semiregular-box template type](#) (`(ranges.semi.wrap)`) which holds a value to iterate over. It would not be possible to reconstruct the exact same range (e.g., iterators pointing to the exact same object) with the semi-regular wrapper. Ranges should have the ability to select this syntax without having it happen to their type implicitly, or as for more information.

This is why there are 4 levels to opt-in to support. If a user defines the 4-argument form which takes a `(tag, range, iterator, sentinel)` quadruplet, then they can use as much information may be available in the range, iterator, and sentinel to reproduce the original range. If they do not need that much information, they can opt-in to the 3-argument version which only takes the `(tag, iterator, sentinel)` triplet. If they want to only deal with the original range and the tag (as an optimization, to save on `reconstruct` potentially calling `ranges::begin(r)` and `ranges::end(r)` on the range and calling the `(tag, range, iterator, sentinel)` version), they can define `reconstruct` for just `(tag, range)`. And, finally, the most brief and sophisticated form is the `(iterator, sentinel)` form, which can reconstruct a range using information from only the iterator and/or sentinel.

If the range opts-in to nothing, then there will always be a default return: `std::ranges::subrange<Iterator, Sentinel>`. This is effectively what any algorithm today already returns (sometimes it uses a hand-made pair type, likely because some iterator-pair usages may require the size to be stored and neither the iterator nor the sentinel can provide the size).

§ 3.4. Applicability

There are many ranges to which this is applicable, but only a handful in the standard library need or satisfy this. If [\[p1391r2\]](#) and [\[p1394r2\]](#) are accepted (they were), then the two most important view types -- `std::span<T, Extent>` and `std::basic_string_view<Char, Traits>` -- will model the *spirit* of the concept (but lack the customization point to

make it generic). `std::ranges::subrange<Iterator, Sentinel, Kind>` already fits this as well. By formalizing concepts in the standard, we can dependably and reliably assert that these properties continue to hold for these ranges. We intend to add a `constexpr friend reconstruct` function to all of the non-`ranges::subrange` types.

There are also upcoming ranges from [\[range-v3\]](#) and elsewhere that could model this concept:

- [\[p1255r4\]](#)'s `std::ranges::ref_maybe_view`;
- [\[p0009r9\]](#)'s `std::mdspan`;
- and, soon to be proposed by this author for the purposes of output range algorithms, [\[range-v3\]](#)'s `ranges::unbounded_view`.

And there are further range adaptor closure objects that could make use of this concept:

- `views::slice`, `views::take_exactly`, `views::drop_exactly` and `views::take_last` from [\[range-v3\]](#)

Note that these changes will greatly aid other algorithm writers who want to preserve the same input ranges. In the future, the standard may provide an `ranges::reconstruct(...)` algorithm for these types.

§ 3.5. Why The Overloads?

It has been questioned why there are four different potential extension points for `ranges::reconstruct`. Indeed, most individuals observe that all of the necessary information required is the range and (possibly) any outstanding iterators. Their version of this paper — termed the "Alternative Design" — looks like this:

```
template <typename Range, typename Iterator, typename Sentinel>
auto reconstruct(Range&& range, Iterator&& iterator, Sentinel&& sentinel);
```

It takes only the range, iterator, and sentinel. There are no tag types. There are no overloads which take: only an iterator and a sentinel; a tag, range, iterator and sentinel; a tag, iterator, and sentinel; or, a tag and a range.

While the user only has to override one in this paper's design, the design above provides **only** one customization point with no room for more or less information. However, many APIs which necessitate various forms of `reconstruct` due to various circumstances when either wrapping certain function / algorithms or dealing with non-generic code within generic code.

§ 3.5.1. Case 0: `std::views::split`

This case justifies `std::ranges::reconstruct(std::in_place_type<R>, iterator, iterator_or_sentinel)`.

The most frequently used way to reconstruct will be the version that takes a tag, an iterator, and another iterator/sentinel. This is motivated by a lot of code, including the code found in the implementation of [\[p1629r1\]](#), [\[ztd.text\]](#). More specifically, `std::views::split_view<V, ...>` stores the range it is made with, `V`, within itself. It does not modify that original range: it only uses iterators from `V` to present the `value_type` / `reference` as it iterates.

[This explanatory post](#) gets into the details, but the most important point is that `std::views::split_view` does not present reconstructed slices of itself using strictly its iterator, its sentinel, and its range. It only presents a view of `(iterator, iterator)`, and completely ignores the sentinel type for the purposes of the presented `reference` type (save for making sure the "current" iteration has not gone beyond-the-end).

This means that the only available information is the **type** of `V`, plus two of iterator objects (and **not** its sentinel object). This is crucial for, e.g. a `zstring` like class, which uses `const char*` and some type of `zstring_sentinel` type:

```
struct zstring_sentinel {
    bool operator==(char const* p) const {
        return *p == '\0';
    }
};

struct zstring : std::ranges::view_interface<zstring> {
```

```

char const* p = nullptr;
constexpr zstring() = default;
constexpr zstring(char const* p) : p(p) { }
constexpr auto begin() const { return p; }
constexpr auto end() const { return zstring_sentinel{}; }

// (1)
friend constexpr std::string_view reconstruct(
    std::in_place_type_t<zstring>,
    const char* f,
    const char* l) noexcept {
    return std::string_view{f, static_cast<std::size_t>(l - f)};
}

// (2)
friend constexpr zstring reconstruct(
    const char* f,
    zstring_sentinel) noexcept {
    return zstring{f};
}
};

```

The reconstruction point under `// (1)` is what gets used by `std::views::split`. We clearly do not want to return a `std::ranges::subrange<...>` type for this, given that the logical substring of a null-terminated string is just a `std::string_view`. Reconstructible helps us accomplish this by letting us have varying customization points for different scenarios. Which, brings us to the next case that continues to serve as justification...

This is required, especially if we would like to ensure that `std::views::split` has optimal behavior.

§ 3.5.2. Case 1: `zstring` and `std::views::drop`

This case justifies `std::ranges::reconstruct(std::in_place_type_t<R>, Range, iterator, iterator_or_sentinel)`, which covers the algorithms present in `[ztd.text]`, and `std::ranges::reconstruct(iterator, iterator_or_sentinel)`, which is the extension point's form needed for `zstring`.

As shown in [Case 0](#), the `zstring` class has a second reconstruction point:

```

struct zstring : std::ranges::view_interface<zstring> {
    // ...

    // (2)
    friend constexpr zstring reconstruct(
        const char* f,
        zstring_sentinel) noexcept {
        return zstring{f};
    }

    // ...
};

```

This reconstruction point is paramount for `views::drop` (and similar algorithms) to work correctly. `views::drop` can be computed by simply incrementing the begin iterator by the provided `min` count passed to the algorithm. So, that means the new `begin()` on the desired `drop_view` is `ranges::begin(E) + min<D>(ranges::size(E), F)`, where `E` is a range, and `ranges::end(E)` is the end. For `zstring`, this is where the second reconstruction point comes in handy. There is no constructor we can use to make `zstring` behave as expected, but with `std::ranges::reconstruct` we can re-constitute the `zstring` properly if `E` is a (potentially cv-qualified) `zstring`:

... Otherwise, if `T` models both `random_access_range` and `sized_range`, [...] then
`ranges::reconstruct(in_place_type<T>, E, ranges::begin(E) + min<D>(ranges::size(E), F),
ranges::end(E))`, except that `E` is evaluated only once.

This will allow us to reconstitute the range. Note that we have 4 pieces here: the tag, range itself `E`, the `iterator` object (advanced by the minimum between the size of the range and `F`), and the `sentinel` object. `zstring`, in this case, does not need the original range itself. But, this is fine: `reconstruct` will drop the `range` type to see if that extension point works, then drop the `tag` object to see if the iterator/sentinel is enough. As it is in `zstring`'s case, that will be the extension point called to reconstitute the `zstring`. This does not mean all ranges can always drop all information given: see [the transform view case](#) for an additional case where being able to pass in the original range actually matters.

§ 3.5.3. Case 2: Legacy Iterator Code

This case justifies `std::ranges::reconstruct(iterator, iterator_or_sentinel)`.

There is too much code in the world that works off of, exclusively, iterators, and does not compose to a range type. Some of them mock semi-range interfaces, such as `UCharIterator` from ICU. Others are just quite literally `std::pair<iterator, iterator>`-like return values and structs with similar information. Whatever the case is, there is a wide body of preexisting code that can benefit from reconstruction even if the range type/range object is not directly available.

There are also many ranges - such as `zstring` above - which genuinely do not need range type information to be reconstructed. Both the iterator and the sentinel pair's type information is enough to strongly identify ranges that are capable of this. Other types of ranges include most ranges in the C++20 ranges library: almost all of them have distinct iterator types paired with either a distinct sentinel type or a `std::default_sentinel_t`. Almost none of these need the original range in order to properly reconstruct and form an unambiguous set given just the iterator and the sentinel. Older `buffer_view<T, Bounds>` class types (the precursor to `std::span` and `std::mdspan`) also have iterators which are distinct to the class itself to traverse the

It does not make sense to cut off all the pre-C++20 iterator-based algorithms and intermediates to not be able to take advantage of ranges code, even if only internally. Modernizing internals is still a useful goal, even if that leads to uses where they are bound to break it apart so that it can fit into a result structure or a `std::pair<It, It>`-like return value from compatibility or API concerns.

It also does not make sense to force a person to need to explicitly specify every single combination of `(range_type, (iterator, sentinel))` in order to be able to properly reconstruct something. For example, if the `std::ranges::reconstruct(iterator, sentinel)` does not exist, then the only way to reconstruct a `zstring` if an algorithm or intermediate step happens to destroy the original range and turn it into a `std::ranges::subrange<const char*, zstring_sentinel>` is to write multiple extension points:

```
struct zstring : std::ranges::view_interface<zstring> {
    // ...

    // (2.1)
    friend constexpr zstring reconstruct(
        std::in_place_type_t<zstring>,
        const char* f,
        zstring_sentinel) noexcept {
        return zstring{f};
    }

    // (2.2)
    friend constexpr zstring reconstruct(
        std::in_place_type_t<std::ranges::subrange<
            const char*, zstring_sentinel
        >>,
        const char* f,
        zstring_sentinel) noexcept {
        return zstring{f};
    }
}
```

```
// ...
};
```

Right now, we are at 2. But as we get more and more ranges that are simple wrappers with some associated behavior, we may need to define even more if we do not have the base-case `iterator`, `sentinel` extension point. This is not a tenable solution to the problem over a long period of time.

§ 3.5.4. Case 3: `transform_view`

This case justifies `std::ranges::reconstruct(std::in_place_type<R>, range, iterator, iterator_or_sentinel)`.

As explained in [p2415] and earlier papers that changed the `std::ranges` model to not always treat all views as cheaply-copied entities, we sometimes need to forward ranges through algorithms or take ranges by reference in order to have more useful information about them and make more intelligent decisions about them. This applies to cases like `std::views::transform_view`, where having access to the original range (as a value or by r-value reference) allows us to perform a much more efficient reconstruction.

```
friend constexpr auto
reconstruct(in_place_type_t<transform_view>, const transform_view& original,
            iterator<iterator_condition> first,
            sentinel<sentinel_condition> last) noexcept(see-below);

friend constexpr auto
reconstruct(in_place_type_t<transform_view>, transform_view&& original,
            iterator<iterator_condition> first,
            sentinel<sentinel_condition> last) noexcept(see-below);
```

The user provides us with a range here, similar to other use cases in the wild (and, perhaps, the majority use case) (for example, most of [ztd.text] consumes its ranges by-value, and can easily afford to move not only the iterator and sentinel but the range that was passed down to it). When the user can give us the original `transform_view`, we can choose between a far more efficient move of not only the guts of the iterator and sentinel, but also of the `original.fun` *exposition-only* member present. This is a key efficiency for more wrapping and nested ranges, and can matter for moving out objects which may allocate (such as a `std::transform_view` with a `std::function<...>`) contained inside. In the case of an l-value reference, we copy.

It is important to note that we only use the four-argument, range-provided form in this extension point here because we do not want to pay the cost of re-forming the `transform_view` when only handed the iterator and the sentinel. Previous versions of this paper had a reconstruction point which used both the `iterator` and the `sentinel` of `transform_view` only, which necessitated copying the original function. While recreating the view, this operation may not be cheap (in both `transform_view`'s case and many other views's cases).

§ 3.6. Alternative Design: `(range, iterator, sentinel)` Reconstruction Point

We could, under a different paradigm that does not take a tag, take the actual range instead and then simply ignore the range argument. Consider, once again, the `zstring` class used in the above Cases:

```
struct zstring : std::ranges::view_interface<zstring> {
    // ...

    // (New Extension: range, iterator, sentinel)
    template <typename Range>
    friend constexpr zstring reconstruct(
        Range&&, // ignored
        const char* f,
        zstring_sentinel) noexcept {
        return zstring{f};
    }
}
```

```
// ...  
};
```

This, however, has a few problems.

Consider `std::views::split`. It does not want to fully reconstruct the `this->base` (exposition-only) range! It only wants to recreate a piece of itself, and keep using `this->base` in future iterations of its reference code. Use of this kind of extension point requires that a range must exist or must be available to pass in. This is **not possible** in the general sense: ranges (and now, views) are no longer required to be default constructible, so we cannot always expect an algorithm to use the original range (or just a default-constructed range, assuming advanced knowledge where you expect the range argument to not be used).

As a base, having the `[iterator, sentinel)` overload is a proper way to signal "I want what is minimally reconstructible from strictly this information, and without paying the cost of potentially rebuilding the entire original range". This is critical in the case of, for example, a follow-on paper which can build on this to request different **types** of ranges be re-created, knowing that only the iterators can be used for such an operation. An example of such can be found for a Future C++ Ranges Proposal in [§ 3.7 Expanding to Owning Containers and More?](#).

§ 3.7. Expanding to Owning Containers and More?

There is a desire to have `reconstruct` automatically apply to things like `std::string` and `std::vector<T>`. For example, it would be nice if `std::views::split` could take in a `std::string` and know to boil it down to `std::string_view` pieces rather than `std::ranges::subrange<impl-defined-iterator, impl-defined-iterator, std::ranges::subrange_kind::sized>`. However, the idea should be swiftly rejected in a generic application like this for a very good reason:

... It is, however, possible that future more refined concepts would no longer be modelled by an input type erased in the above fashion. For this reason Casey Carter argued against changing `views::all` and it was dropped from this proposal.

[The current proposal does not suffer from this uncertainty, because we are preserving the exact input type and there is no erasure.]

— [Hannes Hauswedell, P1739r4](#)

Changes of this magnitude have to be made on a case-by-case basis. For example, for `std::views::split`, a future paper can use a sequence of potential reconstruction steps to try to pick the right reconstruction algorithm. For example, its `find`

[next](#) exposition-only function could use the reconstruction traits in the code here to "select" a potentially better reconstruction point, before simply falling back to the baseline reconstruction call:

```
constexpr auto find-next(iterator_t<V> it); // exposition only
```

Effects: Equivalent to:

```
auto [b, e] = ranges::search(subrange(it, ranges::end(base_)), pattern_);
if (b != ranges::end(base_) && ranges::empty(pattern_)) {
    ++b;
    ++e;
}
if constexpr (ranges::reconstructible_range<V, iterator_t<V>, iterator_t<V>>) {
    return ranges::reconstruct(in_place_type<V>, b, e);
}
else if constexpr (std::is_const_v<ranges::range_value_t<V>>
    && is-char-type<ranges::range_value_t<V>>
    && ranges::reconstructible_range<
        basic_string_view<ranges::range_value_t<V>>,
        iterator_t<V>, iterator_t<V>
    >) {
    using _Tag = in_place_type_t<
        basic_string_view<ranges::range_value_t<V>
    >>;
    return ranges::reconstruct(_Tag(), b, e);
}
else if constexpr (ranges::reconstructible_range<
    span<ranges::range_value_t<V>>,
    iterator_t<V>, iterator_t<V>
>) {
    using _Tag = in_place_type_t<span<ranges::range_value_t<V>>>;
    return ranges::reconstruct(_Tag(), b, e);
}
else {
    return ranges::reconstruct(in_place_type<V>, b, e);
}
```

The above code is not necessarily exactly correct (e.g., one would need to accommodate for instantiation errors in [basic_string_view](#) for improper types, and so carefully restructure the `if constexpr` checks), but overall that is how a future paper might better improve the [value_types](#) for `std::views::split_view`. This paper does not engage in this fix because it focuses purely on reconstruction and not anything else: future work is welcome.

More pointedly, however, this is a choice that has to be made on an algorithm-by-algorithm and view-by-view basis. It is not something that can be **generally** applied to every single view that wishes to attempt reconstruction. This is where the concepts for checking if there exists an ADL-callable version of [reconstruct](#), and why they are put into the standard. As demonstrated in the wording for [take_view](#) and [drop_view](#), the concept can simplify the specification and open it to a wider range of potentially reconstructible types, rather than a few hand-picked ones. Users can also use this for their own algorithms to apply optimizations, as demonstrated in the case of a potentially enhanced [views::split](#) above.

§ 4. Deployment Experience

This change was motivated by Hannes Hauswedell's [\[p1739r4\]](#) and became very important for the ranges work done with text encoding. There is a C++17 implementation [at the ztd.text repository here](#), which is an implementation for the interface needed for [\[p1629r1\]](#). It is meant to solve the deployment issues with P1739's merging into the Standard.

§ 5. Impact

As a feature that is opt-in thanks to the [ranges::reconstruct](#) Customization Point Object design, no currently created range or previously made range outside of the standard library is affected.

Furthermore, this is a new and separate set of concepts. It is not to be added to the base [Range](#) concept, or added to any other concept. It is to be applied separately to the types which can reasonably support it for the benefit of algorithms and code which can enhance the quality of their implementation.

Finally, Hannes Hauswedell's [\[p1739r4\]](#) with the explicit intention to mark certain ranges as reconstructible by hardcoding their behavior into the standard and come back with an opt-in fix during the C++23 cycle. This paper completes that promise.s

§ 6. Proposed Changes

The following wording is relative to the latest C++ Draft paper.

§ 6.1. Feature Test Macro

This paper results in a concept to help guide the further development of standard ranges and simplify their usages in generic contexts. There is one proposed feature test macro, `__cpp_lib_reconstructible_range`, which is to be input into the standard and then explicitly updated every time a `reconstruct` from a is added to reflect the new wording. We hope that by putting this in the standard early, most incoming ranges will be checked for compatibility with `iterator_reconstructible_range` and `reconstructible_range`.

§ 6.2. Intent

The intent of this wording is to provide greater generic coding guarantees and optimizations by allowing for a class of ranges and views that model the new exposition-only definitions of a reconstructible range:

- add a new feature test macro for reconstructible ranges to cover constructor changes;
- add a new customization point object for `ranges::reconstruct`;
- add the customization point to many different ranges that can be reconstructed (`transform_view`, `split_view`, `subrange`, `basic_string_view`, `span`, `iota_view`);
- add three new concepts to `[range.req]`;
- and, use the concept in a few algorithms to return better versions of themselves (`drop_view` and `take_view`).

§ 6.3. Proposed Library Wording

§ 6.3.1. Add a feature test macro `__cpp_lib_reconstructible_range`.

Editor's Note: Substitute appropriate value if plenary accepted.

```
#define __cpp_lib_reconstructible_range 2022MML // also in <ranges>
```

§ 6.3.2. Insert into §24.2 Header `<ranges>` Synopsis [\[ranges.syn\]](#) a new customization point object in the inline namespace:

```
namespace std::ranges {
    inline namespace unspecified {
    ...

    inline constexpr unspecified reconstruct = unspecified;

    ...
    }
    ...

    template <class It, class Sen = It>
        using iterator_reconstruct_t = decltype(ranges::reconstruct(
            declval<It>(), declval<Sen>().
        ));

    template <class T,
        class It = ranges::iterator_t<T>,
        class Sen = ranges::sentinel_t<T>
        using reconstruct_t = decltype(ranges::reconstruct(
            in_place_type<T>, declval<It>(), declval<Sen>().
        ));

    template <class T,
        class Tag = remove_cvref_t<T>,
        class It = ranges::iterator_t<T>,
        class Sen = ranges::sentinel_t<T>>
        using range_iterator_reconstruct_t = decltype(ranges::reconstruct(
            in_place_type<Tag>, declval<T>(), declval<It>(), declval<Sen>().
        ));

    ...

    template <class It, class Sen = It>
        concept iterator_reconstructible_range = see below;

    template <class Tag,
        class It = ranges::iterator_t<R>,
        class Sen = ranges::sentinel_t<R>>
        concept reconstructible_range = see below;

    template <class R, class Tag = remove_cvref_t<R>,
        class It = ranges::iterator_t<R>,
        class Sen = ranges::sentinel_t<R>>
        concept range_iterator_reconstructible_range = see below;

    ...
    }
```

6.3.3. Insert into §24.4.2 Ranges `[range.range]`'s after paragraph 7, one additional paragraph:

⁸The concepts `iterator_reconstructible_range`, `reconstructible_range`, and `range_iterator_reconstructible_range` concepts describe the requirements on ranges that are efficiently constructible from certain iterator and sentinel types, alongside possible additional information from the range itself.

```
template <class It, class Sen = It>
    concept iterator_reconstructible_range =
        (is_class_v<It> || is_class_v<Sen>
         || is_enum_v<It> || is_enum_v<Sen>)
        && requires (It first, Sen last) {
            reconstruct(
                forward<It>(first),
                forward<Sen>(last)
            );
        };

template <class Tag,
    class It = ranges::iterator_t<R>,
    class Sen = ranges::sentinel_t<R>>
    concept reconstructible_range =
        ranges::range<R>
        && (iterator_reconstructible_range<It, Sen>
        || requires (It first, Sen last) {
            reconstruct(
                in_place_type<Tag>,
                move(first),
                move(last)
            );
        });

template <class R, class Tag = remove_cvref_t<R>,
    class It = ranges::iterator_t<R>,
    class Sen = ranges::sentinel_t<R>>
    concept range_iterator_reconstructible_range =
        ranges::range<R>
        && (reconstructible_range<Tag, It, Sen>
        || requires (R range, It first, Sen last) {
            reconstruct(
                in_place_type<Tag>,
                forward<R>(range),
                forward<It>(first),
                forward<Sen>(last)
            );
        });
```

⁹Let `r` be a range with type `R`, `T` be some type, `i` be an iterator of type `I` and `s` be a sentinel of type `S`.

9.1 — Let `it Sen re_range` be the result of `ranges::reconstruct(i, s)` if `I` and `S` satisfy `ranges::iterator_reconstructible_range`. `r` models `ranges::iterator_reconstructible_range` if

— `i == ranges::begin(it Sen re_range)` is true, and
 — `s == ranges::end(it Sen re_range)` is true.

9.2 — Let `re_range` be the result of `ranges::reconstruct(in_place_type<remove_cvref_t<T>>, i, s)` if `remove_cvref_t<T>`, `I`, and `S` satisfy `ranges::iterator_reconstructible_range`. `r` models `ranges::reconstructible_range` if

— `i == ranges::begin(re_range)` is true, and
 — `s == ranges::end(re_range)` is true.

9.3 — Let `range_it re_range` be the result of `ranges::reconstruct(in_place_type<remove_cvref_t<T>>, r, i, s)`, if `R`, `remove_cvref_t<T>`, `I`,

and `S` satisfy `ranges::range_iterator_reconstructible_range`. Then `range_it_re_range` models `ranges::range_iterator_reconstructible_range` if

— `i == ranges::begin(range_it_re_range)` is true, and
— `s == ranges::end(range_it_re_range)` is true.

[Note: If an iterator and a sentinel is passed in that does not come directly from a `ranges::begin` or `ranges::end` expression, then the range of the reconstructed range matches the given iterator and sentinel so long as it is appropriately related. — end Note]

6.3.4. Insert a new sub-clause "§24.3.13 `ranges::reconstruct` [`range.prim.recons`]", after "§24.3.12 `ranges::cdata` [`range.prim.cdata`]":

24.3.12 `ranges::reconstruct`

[`range.prim.recons`]

¹ The name `ranges::reconstruct` denotes a customization point object.

² The expression `ranges::reconstruct(I, S)` for some sub-expressions `I` and `S` is expression-equivalent to:

(2.1) `reconstruct(std::forward<decltype(I)>(I), std::forward<decltype(S)>(S))` if either `remove_cvref_t<decltype(I)>` or `remove_cvref_t<decltype(S)>` are class or enumeration types, it is a valid expression, and both `decltype(I)` and `decltype(S)` model `iterator_reconstructible_range`.

(2.2) Otherwise, `ranges::subrange<remove_cvref_t<decltype(I)>, remove_cvref_t<decltype(S)>>(std::forward<decltype(I)>(I), std::forward<decltype(S)>(S)))` if it is a valid expression.

(2.3) Otherwise, `ranges::reconstruct(I, S)` is ill-formed.

³ The expression `ranges::reconstruct(in_place_type<R>, I, S)` for some type `R` and some sub-expressions `I` and `S` is expression-equivalent to:

(2.1) `reconstruct(in_place_type<R>, std::forward<decltype(I)>(I), std::forward<decltype(S)>(S))` if it is a valid expression and `R`, `decltype(I)`, and `decltype(S)` model `reconstructible_range`.

(2.2) Otherwise, `ranges::reconstruct(std::forward<decltype(I)>(I), std::forward<decltype(S)>(S))`.

⁴ The expression `ranges::reconstruct(in_place_type<R>, SR, I, S)` for some type `R`, and some sub-expressions `SR`, `I`, and `S` is expression-equivalent to:

(2.1) `reconstruct(in_place_type<R>, std::forward<decltype(SR)>(SR), std::forward<decltype(I)>(I), std::forward<decltype(S)>(S))` if it is a valid expression and `decltype(SR)`, `R`, `decltype(I)`, and `decltype(S)` model `range_iterator_reconstructible_range`.

(2.2) Otherwise, `ranges::reconstruct(in_place_type<R>, std::forward<decltype(I)>(I), std::forward<decltype(S)>(S))`.

6.3.5. Add to "§21.4.3.1 **General**" [`string.view.template.general`] a **friend** function ADL extension point for `basic_string_view`:

```
// [string.view.reconstruct]
template <class It, class End>
    friend constexpr basic_string_view
        reconstruct(in_place_type_t<basic_string_view>
            It first, End last) noexcept;
```

§ 6.3.6. Add a new subclause "§21.4.◆◆◆◆ Range Reconstruction" [string.view.reconstruct] in §21.4:

21.4.◆◆◆◆ Range Reconstruction

[string.view.reconstruct]

```
template <class It, class End>
    friend constexpr basic_string_view
    reconstruct(in_place_type_t<basic_string_view>
        It first, End last) noexcept;
```

¹ Constraints: Let *U* be `remove_reference_t<iter_reference_t<It>>`.

(1.1) — `is_convertible_v<U(*)[], const_value_type(*)[]>` is true. [Note: The intent is to allow only qualification conversions of the iterator reference type to element type. — end note]

(1.2) — It models `contiguous_iterator`.

(1.3) — End models `sized_sentinel_for<It>`.

² Returns: `basic_string_view(std::move(first), std::move(last))`.

§ 6.3.7. Add to "§22.7.3.1 Overview" [span.overview] a **friend** function ADL extension point for `span`:

```
// [span.reconstruct]
template <class It, class End>
    friend constexpr auto
    reconstruct(in_place_type_t<span>, It first, End last) noexcept;
```

§ 6.3.8. Add a new subclause "§22.7.3.◆◆◆◆ Range Reconstruction" [span.reconstruct] in §22.7.3:

22.7.3.◆◆◆◆ Range Reconstruction

[span.reconstruct]

```
template <class It, class End>
    friend constexpr auto
    reconstruct(in_place_type_t<span>, It first, End last) noexcept;
```

¹ Constraints: Let *U* be `remove_reference_t<iter_reference_t<It>>`.

(1.1) — `is_convertible_v<U(*)[], element_type(*)[]>` is true. [Note: The intent is to allow only qualification conversions of the iterator reference type to element type. — end note]

(1.2) — It models `contiguous_iterator`.

(1.3) — End models `sized_sentinel_for<It>`.

² Returns: `span<ElementType>(std::move(first), std::move(last))` or a span of static extent using `first` and `last`. See *Remarks*.

³ Remarks: The return type may be promoted to a span with a static extent if the implementation is capable of deriving such information from the given iterator and sentinel.

§ 6.3.9. Add to "§24.5.4.1 **General**" [[range.subrange.general](#)] a **friend** function ADL extension point for [subrange](#):

```
friend constexpr subrange
reconstruct(in_place_type_t<subrange>, I first, S last) noexcept(see-below);

friend constexpr auto
reconstruct(in_place_type_t<subrange>, I first, S last) noexcept(see-below);
```

§ 6.3.10. Add a new subclause "§24.5.4. **Range Reconstruction**" [[range.subrange.reconstruct](#)] in §24.5.4:

22.7.3. **Range Reconstruction**

[[range.subrange.reconstruct](#)]

```
friend constexpr subrange
reconstruct(in_place_type_t<subrange>, I first, S last) noexcept(see-below);
```

¹ Returns: `subrange(std::move(first), std::move(last))`.

² Throws: Anything from evaluating the *Returns*. Otherwise, nothing.

```
friend constexpr auto
reconstruct(in_place_type_t<subrange>, I first, I last) noexcept(see-below);
```

³ Returns: `subrange<I>(std::move(first), std::move(last))`.

⁴ Throws: Anything from evaluating the *Returns*. Otherwise, nothing.

§ 6.3.11. Add to "§24.6.4.2 Class template [iota_view](#)" [[range.iota](#)] a **friend** function ADL extension point for [iota_view](#):

```
template <class S>
friend constexpr iota_view
reconstruct(iterator first, S last) noexcept(see-below);
```

§ 6.3.12. Add one new paragraph to "§24.6.4.2 Class template [iota_view](#)" [[range.iota](#)]:

```
template <class S>
friend constexpr auto
reconstruct(iterator first, S last) noexcept(see-below);
```

◆◆¹ Constraints: *S* is *iterator* or *S* is *sentinel*.

◆◆² Effects: Equivalent to:

```
if constexpr (same_as<iterator, S>) {
    return iota_view<decltype(*std::move(first)), decltype(*std::move(last))>(
        std::move(first), *std::move(last)
    );
}
else {
    return iota_view(std::move(first), std::move(last));
}
```

◆◆³ Throws: Anything from evaluating the *Effects*. Otherwise, nothing.

- § 6.3.13. Add to "§24.6.2.2 Class template `empty_view`" [[range.empty.view](#)] a **friend** function ADL extension point for `empty_view`:

```
friend constexpr empty_view  
reconstruct(in_place_type_t<empty_view>, T*, T*) noexcept {  
    return empty_view{};  
}  
  
friend constexpr empty_view  
reconstruct(in_place_type_t<empty_view>, nullptr_t, T*) noexcept {  
    return empty_view{};  
}  
  
friend constexpr empty_view  
reconstruct(in_place_type_t<empty_view>, T*, nullptr_t) noexcept {  
    return empty_view{};  
}  
  
friend constexpr empty_view  
reconstruct(in_place_type_t<empty_view>, nullptr_t, nullptr_t) noexcept {  
    return empty_view{};  
}
```

- § 6.3.14. Add to "§24.7.6.2 Class template `transform_view`" [[range.transform.view](#)] a **friend** function ADL extension point for `transform_view`:

```
template <bool iterator_condition, bool sentinel_condition>  
friend constexpr auto  
reconstruct(in_place_type_t<transform_view>, const transform_view original,  
    iterator<iterator_condition> first,  
    sentinel<sentinel_condition> last) noexcept(see-below);  
  
template <bool iterator_condition, bool sentinel_condition>  
friend constexpr auto  
reconstruct(in_place_type_t<transform_view>.,  
    transform_view&& original,  
    iterator<iterator_condition> first,  
    sentinel<sentinel_condition> last) noexcept(see-below);
```

6.3.15. Add new paragraphs to "§24.6.4.2 Class template `transform_view`" [[range.transform](#)]:

```
template <bool iterator_condition, bool sentinel_condition>
    friend constexpr auto
    reconstruct(in_place_type_t<transform_view>,
               const transform_view& original,
               iterator<iterator_condition> first,
               sentinel<sentinel_condition> last) noexcept(see-below);
```

◆◆¹ *Constraints:* `V` models `reconstructible_range` and `copy_constructible`. `F` models `copy_constructible`.

◆◆² *Returns:* `transform_view(ranges::reconstruct(std::in_place_type<V>, std::move(first).current, std::move(last).end), *original.fun)`.

◆◆³ *Throws:* Anything from evaluating the *Returns*. Otherwise, nothing.

```
template <bool iterator_condition, bool sentinel_condition>
    friend constexpr auto
    reconstruct(in_place_type_t<transform_view>, transform_view&& original,
               iterator<iterator_condition> first,
               sentinel<sentinel_condition> last) noexcept(see-below);
```

◆◆⁴ *Constraints:* `V` models `reconstructible_range`.

◆◆⁵ *Returns:* `transform_view(ranges::reconstruct(std::in_place_type<V>, std::move(first).current, std::move(last).end), std::move(*original.fun))`.

◆◆⁶ *Throws:* Anything from evaluating the *Returns*. Otherwise, nothing.

§ 6.3.16. Modify "§24.7.7.1 Overview " [[range.take.overview](#)] for `views::take`'s paragraph 2:

² The name `views::take` denotes a range adaptor object ([`range.adaptor.object`]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::take(E, F)` is ill-formed. Otherwise, the expression `views::take(E, F)` is expression-equivalent to:

(2.1) — If `T` is a specialization of `ranges::empty_view` ([`range.empty.view`]), then `((void) F, decay-copy(E))`:

(2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is

(2.2.1) — a specialization of `span` ([`views.span`]) where `T::extent == dynamic_extent`;

(2.2.2) — a specialization of `basic_string_view` ([`string.view`]);

(2.2.3) — a specialization of `ranges::iota_view` ([`range.iota.view`]), or

(2.2.4) — a specialization of `ranges::subrange` ([`range.subrange`]);

then `T{ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F)}`, except that `E` is evaluated only once.

(2.3) — Otherwise, `ranges::take_view{E, F}`:

(2.1) — If `T` is a specialization of `ranges::empty_view` ([`range.empty.view`]), then `((void) F, decay-copy(E))`.

(2.2) — Otherwise, if `T` models both `random_access_range` and `sized_range` and, `T.remove_cvref_t<T>`, `ranges::iterator_t<T>`, and `ranges::iterator_t<T>` model `range_iterator_reconstructible_range`, then `ranges::reconstruct(in_place_type<T>, E, ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F))`, except that `E` is evaluated only once.

(2.3) — Otherwise, `ranges::take_view{E, F}`.

6.3.17. Modify "§24.7.9.1 Overview" [range.drop.overview] for `views::drop`'s paragraph 2:

² The name `views::drop` denotes a range adaptor object ([range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::drop(E, F)` is ill-formed. Otherwise, the expression `views::drop(E, F)` is expression-equivalent to:

(2.1) ~~If `T` is a specialization of `ranges::empty_view` ([range.empty.view]), then `((void) F, decay-copy(E))`:~~

(2.2) ~~Otherwise, if `T` models `random_access_range` and `sized_range` and is~~

~~(2.2.1) a specialization of `span` ([views.span]) where `T::extent == dynamic_extent`;~~

~~(2.2.2) a specialization of `basic_string_view` ([string.view]);~~

~~(2.2.3) a specialization of `ranges::iota_view` ([range.iota.view]), or~~

~~(2.2.4) a specialization of `ranges::subrange` ([range.subrange]);~~

~~then `T{ranges::begin(E) + min<D>(ranges::size(E), F), ranges::end(E)}`, except that `E` is evaluated only once.~~

(2.3) ~~Otherwise, `ranges::drop_view{E, F}`:~~

(2.1) — If `T` is a specialization of `ranges::empty_view` ([range.empty.view]), then `((void) F, decay-copy(E))`.

(2.2) — Otherwise, if `T` models both `random_access_range` and `sized_range`, and `T.remove_cvref_t<T>`, `ranges::iterator_t<T>`, and `ranges::sentinel_t<T>` model `range_iterator_reconstructible_range`, then `ranges::reconstruct(in_place_type<T>, E, ranges::begin(E) + min<D>(ranges::size(E), F), ranges::end(E))`, except that `E` is evaluated only once.

(2.3) — Otherwise, `ranges::drop_view{E, F}`.

6.3.18. Modify "§24.7.14.2 Class template `split_view`" [range.split.view] for `split_view`'s *find-next* exposition-only function:

```
constexpr subrange<iterator_t<V>> find-next(iterator_t<V>); // exposition only
using re_type = reconstruct<T<V>, iterator_t<V>, iterator_t<V>>; // exposition only
constexpr re_type find-next(iterator_t<V>); // exposition only
```

6.3.19. Modify "§24.7.14.3 Class `split_view::iterator`" [range.split.iterator] for `split_view<...>::iterator`'s *value_type* in the Synopsis:

```
template <...>
    requires ...
class split_view<...>::iterator {
private:
    //...
    subrange<iterator_t<V>> next_ = subrange<iterator_t<V>>(); // exposition only
public:
    //...
    using value_type = subrange<iterator_t<V>>;
    //...
};
```

```

template <...>
    requires ...
class split_view<...>::iterator {
private:
    // ...
    re_type next = re_type(); // exposition only
public:
    // ...
    using value_type = re_type;
    // ...
};

```

§ 6.3.20. Modify "§24.7.14.2 Class template `split_view`" [range.split.view] for `split_view`'s Overview and paragraph 5:

```
constexpr subrange<iterator_t<V>> find_next(iterator_t<V> it); // exposition only
```

Effects : Equivalent to:

```

auto [b, e] = ranges::search(subrange(it, ranges::end(base_)), pattern_);
if (b != ranges::end(base_) && ranges::empty(pattern_)) {
    ++b;
    ++e;
}
return {b, e};

```

```
constexpr re_type find_next(iterator_t<V> it); // exposition only
```

Effects : Equivalent to:

```

auto [b, e] = ranges::search(subrange(it, ranges::end(base_)), pattern_);
if (b != ranges::end(base_) && ranges::empty(pattern_)) {
    ++b;
    ++e;
}
return ranges::reconstruct(in_place_type<V>, b, e);

```

§ 7. Acknowledgements

Thanks to Corentin Jabot, Christopher DiBella, and Hannes Hauswedell for pointing me to [p1035r7] and [p1739r4] to review both papers and combine some of their ideas in here. Thanks to Eric Niebler for prompting me to think of the generic, scalable solution to this problem rather than working on one-off fixes for individuals views.

Thank you to Oktal, Anointed of ADL, Blessed Among Us, and Morwenn, the ever-watching Code Guardian for suggesting improvements to the current concept form.

§ References

§ Informative References

[BLOG-VIEWS-SPLIT]

JeanHeyd Meneide. *views::split: The Final Frontier*. December 12th, 2021. URL: <https://thephd.dev/ranges-split-final-frontier-reconstructible>

[ICUCHARITERATOR]

Unicode Consortium; International Components for Unicode. *CharacterIterator: ICU Documentation*. March 22nd, 2021. URL: <https://unicode-org.github.io/icu/userguide/strings/characteriterator.html>

[N4835]

Richard Smith. *Working Draft, Standard for Programming Language C++*. 8 October 2019. URL: <https://wg21.link/n4835>

[P0009R9]

H. Carter Edwards, Bryce Adelstein Lelbach, Daniel Sunderland, D. S. Hollman, Christian Trott, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Mark Hoemmen. *mdspan: A Non-Ownning Multidimensional Array Reference*. 20 January 2019. URL: <https://wg21.link/p0009r9>

[P1035R7]

Christopher Di Bella, Casey Carter, Corentin Jabot. *Input Range Adaptors*. 2 August 2019. URL: <https://wg21.link/p1035r7>

[P1255R4]

Steve Downey. *A view of 0 or 1 elements: view::maybe*. 17 June 2019. URL: <https://wg21.link/p1255r4>

[P1391R2]

Corentin Jabot. *Range constructor for std::string_view*. 17 June 2019. URL: <https://wg21.link/p1391r2>

[P1394R2]

Corentin Jabot, Casey Carter. *Range constructor for std::span*. 17 June 2019. URL: <https://wg21.link/p1394r2>

[P1629R1]

JeanHeyd Meneide. *Transcoding the world - Standard Text Encoding*. 2 March 2020. URL: <https://wg21.link/p1629r1>

[P1739R4]

Hannes Hauswedell. *Avoid template bloat for safe_ranges in combination with 'subrange-y' view adaptors*. 1 March 2020. URL: <https://wg21.link/p1739r4>

[P1989R0]

Corentin Jabot. *Range constructor for std::string_view 2: Constrain Harder*. 25 November 2019. URL: <https://wg21.link/p1989r0>

[P2210R2]

Barry Revzin. *Superior String Splitting*. 5 March 2021. URL: <https://wg21.link/p2210r2>

[P2415]

Barry Revzin; Tim Song. *What is a View?*. July 13th, 2021. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2415r0.html>

[RANGE-V3]

Eric Niebler; Casey Carter. *range-v3*. June 11th, 2019. URL: <https://github.com/ericniebler/range-v3>

[RANGE-V3-1192]

ThePhD. *Ranges which take a sentinel should be constructible from {Iterator, Sentinel}*. June 11th, 2019. URL: <https://github.com/ericniebler/range-v3/issues/1192>

[ZTD.TEXT]

JeanHeyd Meneide; Shepherd's Oasis, LLC. *ztd.text - A C++ Text Library*. November 20th, 2021. URL: <https://ztdtext.readthedocs.io/en/latest/>